Variables: Names, Address, Type, Scope

Programming Languages Intro

Dr. Igor Steinmacher

NAU NORTHERN ARIZONA UNIVERSITY

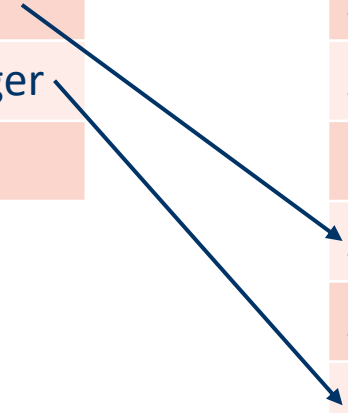
Variables

- Abstraction for memory cells

- Characteristics
  - Name
  - Address
  - Value
  - Type
  - Scope

name	address	type (?)
x	A10	char
y	A12	integer
	…	

address	value
A01	A10
A02	A12
…	…
A10	‘a’
A11	99.99
A12	7
…	…



Names

- Used to identify different entities
  - Variables
  - Functions
  - Types

- Each languages has its own rules for naming identifiers
  - Lexical rules for names.
  - Collection of reserved words or keywords.
  - Case sensitivity
  - Symbols

PHP → Variables start with $
Perl → $, @, % represent the type
FORTRAN I → maximum length 6
COBOL → maximum length: 30

Reserved words

- Cannot be used as *Identifiers*
- Usually identify major constructs: *if while switch*
- Predefined identifiers: e.g., library routines
  - can be redefined

Value and Address

- *l-value* → use of a variable name to denote its address
- *r-value* → use of a variable name to denote its value

```
x = y + 1
```

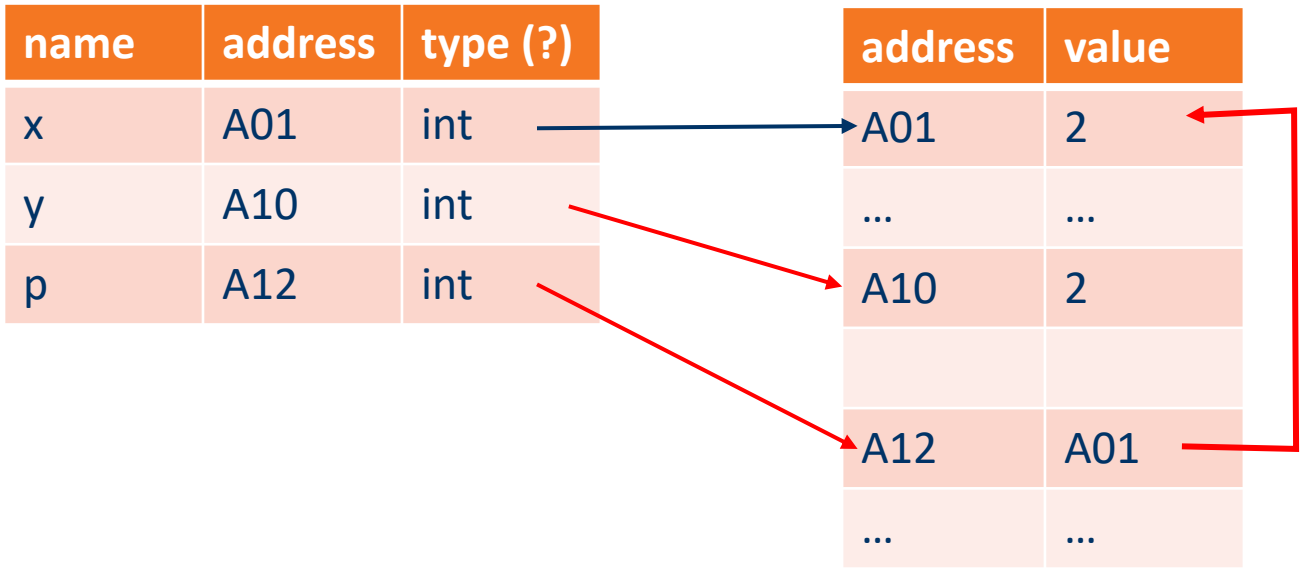
l-value: I need your address so I can store the result

r-value: give me your value, I need to calculate the expression

Dereferencing

Some languages support dereferencing

```
int x = 5;
int y;
int *p;
p = &x;
*p = 2;
y = *p;
```



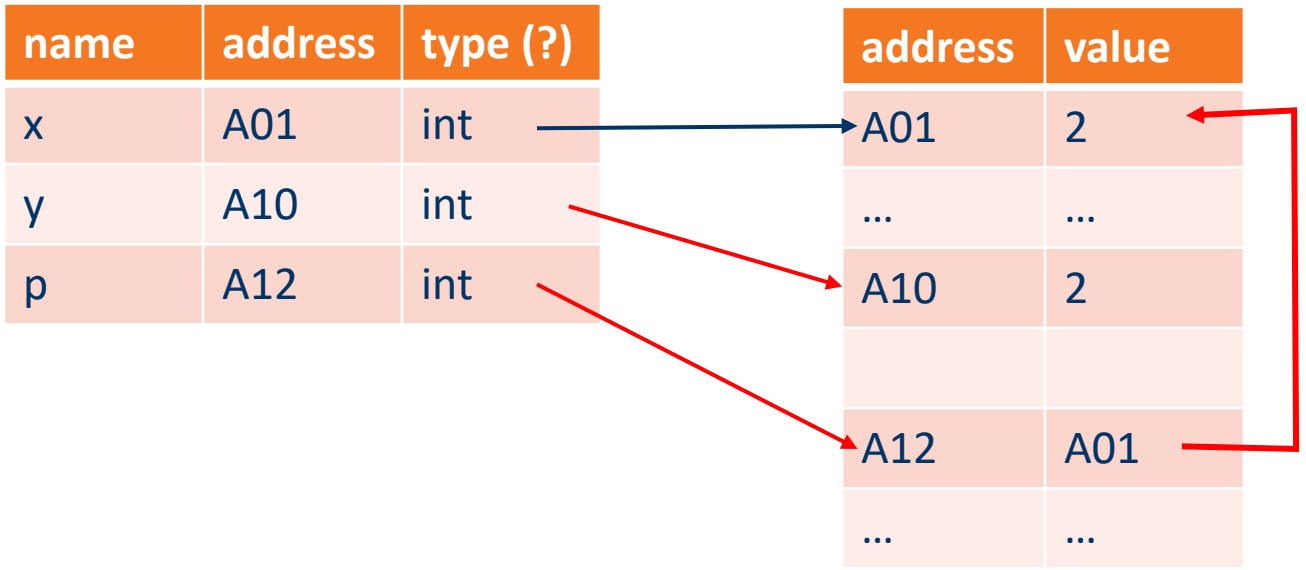
name	address	type (?)
x	A01	int
y	A10	int
p	A12	int

address	value
A01	2
…	…
A10	2
A12	A01
…	…

Dereferencing

Some languages support dereferencing

```
int x = 5;
int y;
int *p;
p = &x;
*p = 2;
y = *p;
```



name	address	type (?)
x	A01	int
y	A10	int
p	A12	int

address	value
A01	2
…	…
A10	2
A12	A01
…	…

Type: Implicit vs. Explicit declaration

- Explicit Declaration:
  - a program statement used for declaring the types of variables

```
int id;
char* name;
```

- Implicit declaration
  - default mechanism for specifying types of variables

  - FORTRAN
    - variables beginning with I-N are assumed to be of type integer
  - Perl

```
$name
@list
%hashmap
```

Type: Static vs. Dynamic Binding

- **Static type binding**
  - Types are explicitly defined (e.g., C, Java)
  - Compiling time

```
int id;
char* name;
```

- **Dynamic type binding**
  - Types are defined by the value that the variable receives (e.g., Python, PHP)
  - Different from weakly typed

```
salary = 100
salary = 100.24
name = 'George'
numbers = 2
numbers = [1, 5, 9]
```

Scope

- The scope of a **name**
  - Collection of statements which can access the name binding
- A variable is **visible** in a statement if it can be referenced in that statement
- **Local** variable is local in a block if it is declared there
- **Non-local** variable is visible within the block but are not declared there

Static Scoping

- **name** is bound to a collection of statements according to its position in the source program
  - Goes according to the nested structure of the program
- Most modern languages use static (or *lexical*) scoping
- Defined in compilation time (scope pushed/popped

```
int x, y, z;

void functionA () {
    int x;
    int y = 7;
    x = y + z;
    …
}
void functionB () {
    int y;
    int z = 2;
    y = 0;
    x = y + z;
    for (int x =0; x<5; x++){
       y = x+1;
    } //y = 5 at the end
    functionA();
}
```

y: local (7)
x: local
z: global (3)
Change the local x

Change x from the global scope
x = 0 + 2 → x = 2

```
void main (){
    x = 1;
    y = 2;
    z = 3;
    functionB ();
}
```

At the end:
x = 2
y = 2
z = 3

Dynamic Scoping

- **name** is bound to its most recent declaration based on the program's **call history**
- Symbol table for each scope is managed at run time
- Scope pushed/popped on stack when entered/exited
- Used be early Lisp, APL, Snobol, Perl.

```
int x, y, z;

void functionA () {
   int x;
   int y = 7;
   x = y + z;
   …
}
void functionB () {
   int y;
   int z = 2;
   y = 0;
   x = y + z;
   for (int x =0; x<5; x++){
      y = x+1;
   } //y = 5 at the end
   functionA();
}
```

y: local = 7
x: local
z: from caller
functionB =2
Change the local x

Change x from the global scope (caller)
x = 0 + 2 → x = 2

```
void main (){
   x = 1;
   y = 2;
   z = 3;
   functionB ();
}
```

At the end:
x = 2
y = 2
z = 3

See you in our next Lesson!

NAU NORTHERN ARIZONA UNIVERSITY